



DOCUMENTO DE PLANIFICACIÓN ÁGIL

Sprint 1 — Front-End

Proyecto: Integrador2_Proyecto — ERP de Reclutamiento

Semana: Planificación y Front-End — Semana 2

Stack tecnológico: Front-End: Next.js · Back-End: FastAPI (Sprint 2) · Base de datos: PostgreSQL

Herramienta de gestión: Github

Elaborado por: Grupo 01

Evaluación y selección de herramientas

Se definieron seis criterios con peso porcentual (suma 100 %) y se compararon dos alternativas de stack completo: la propuesta del equipo (Next.js + FastAPI + PostgreSQL) frente a la línea base institucional sugerida en la guía (Angular + Spring Boot + PostgreSQL). Cada alternativa se calificó de 1 a 5 por criterio; el puntaje ponderado es calificación $\times$ peso.

Criterio	Peso	Alt. A — Next.js + FastAPI + PostgreSQL	Punt. A	Alt. B — Angular + Spring Boot + PostgreSQL	Punt. B
Compatibilidad	25 %	5	1.25	4	1.00
Competencia del equipo	20 %	5	1.00	2	0.40
Mantenibilidad / soporte	20 %	4	0.80	5	1.00
Seguridad / actualización	15 %	4	0.60	4	0.60
Automatización / pruebas	10 %	4	0.40	4	0.40
Costo / licencias	10 %	5	0.50	4	0.40
Total	100 %		4.55		3.80

DECISIÓN:

La Alternativa A (Next.js + FastAPI + PostgreSQL) obtiene el mayor puntaje ponderado (4.55 sobre 3.80), principalmente por la competencia real del equipo en JavaScript/TypeScript y Python. Se adopta esta alternativa. El equipo decide además evolucionar la arquitectura de Back-End hacia microservicios (uno por dominio: postulantes, empresas/vacantes y procesos de selección) en lugar del monolito modular sugerido como línea base, para permitir que cada integrante avance su módulo de forma independiente; se conservan de la línea base institucional GitHub Flow, Docker Compose y Figma, que no dependen del estilo arquitectónico elegido.

Línea tecnológica de referencia adoptada por el equipo

Capa	Decisión final	Motivo técnico y operativo
Front-End	Next.js 16 + TypeScript + Node.js 20 LTS (organizado en múltiples micro-aplicaciones y servicios frontales)	Componentización avanzada, tipado estricto y despliegue desacoplado mediante contenedores Docker dedicados.

Capa	Decisión final	Motivo técnico y operativo
Back-End	FastAPI (Python) organizado en Microservicios por dominio	Desarrollo ágil de API REST, validación con Pydantic, documentación automática OpenAPI y aislamiento por dominio funcional.
Datos	PostgreSQL (bases o esquemas independientes por microservicio)	Modelo relacional robusto, integridad referencial y soporte transaccional desacoplado.
Arquitectura	Microservicios + API REST con API Gateway/BFF	Alineado con la distribución de historias por integrante (ADR-002), permitiendo escalabilidad y desarrollo paralelo autónomo.
Desarrollo e Infraestructura	Docker Compose + VS Code + GitHub Flow	Orquestación local multicontenedor para servicios frontales (erp, anuncios) y base de datos, garantizando reproducibilidad total.

Estructura del repositorio

Ofrecer una interfaz Front-End funcional en Next.js que permita a reclutadores y administradores autenticarse, gestionar postulantes, empresas clientes y requerimientos, y dar seguimiento al proceso de selección, quedando lista para integrarse con el Back-End en FastAPI y la base de datos PostgreSQL en el Sprint 2.

Carpeta / Archivo	Contenido y Estructura
.github/	Plantillas de issues, pull requests y workflows de integración continua (lint y build).
backend/	Estructura de microservicios FastAPI por dominio y API Gateway/BFF (Sprint 2).
frontend/ERP/ y frontend/ANUNCIOS/	Módulos de Front-End contenerizados de forma independiente en la infraestructura local.
docs/decisiones/	Registros formales ADR-001 y ADR-002.
compose.yaml	Configuración Docker Compose multicontenedor para servicios frontales, backend y base de datos PostgreSQL.
CONTRIBUTING.md · README.md	Guías de colaboración y puesta en marcha del repositorio.

Tablero del equipo (GitHub Projects)

El tablero se organiza con las columnas Backlog, Ready, In progress, In review y Done, y los campos Status, Priority, Size, Iteration, Requirement ID y Owner, acordando un límite de un ítem en progreso por persona para evitar trabajo fragmentado. Este tablero es el mismo mostrado en la sección 12 del documento de la Semana 1, ahora enlazado a issues y pull requests reales del repositorio.

Reproducibilidad y Entorno Local (Docker Compose con Microservicios Frontales)

El entorno de desarrollo se levanta de manera reproducible mediante Docker Compose, gestionando de forma aislada los contenedores del Front-End y del motor de base de datos PostgreSQL [cite: 10]. El archivo `infra/compose.yaml` define los servicios multi-contenedor de la siguiente manera:

services:

erp:

container_name: erp-service

build:

context: ./frontend/ERP

dockerfile: Dockerfile

ports:

- "3000:3000"

volumes:

- ./frontend/ERP:/app

- /app/node_modules

- /app/.next

environment:

- WATCHPACK_POLLING=true

- NODE_ENV=development

anuncios:

container_name: anuncios-service

build:

context: ./frontend/ANUNCIOS

dockerfile: Dockerfile

ports:

- "3001:3001"

volumes:

- ./frontend/ANUNCIOS:/app

- /app/node_modules

- /app/.next

environment:

- WATCHPACK_POLLING=true

- NODE_ENV=development

Comandos de inicialización y verificación local:

`cp infra/.env.example infra/.env` # Configurar credenciales seguras

`docker compose --env-file infra/.env -f infra/compose.yaml up -d`

`docker compose --env-file infra/.env -f infra/compose.yaml ps`

Del requerimiento a la experiencia de usuario (Flujo Funcional)

Historia priorizada para el flujo de esta semana: **HU-04** — «*Como reclutador, quiero listar y buscar postulantes por distintos filtros para ubicar candidatos rápidamente*».

Flujo prioritario

Paso	Intención del usuario	Respuesta del sistema	Estado alterno / Excepción
1	Ingresar a la sección de postulantes del ERP.	Carga el listado paginado inicial de candidatos.	Error de conexión / Datos incompletos.
2	Ingresar un criterio de búsqueda (DNI, nombre, especialidad).	Filtra la tabla de forma reactiva en menos de 500 ms.	Sin resultados coincidentes para el filtro.
3	Aplicar filtros adicionales por estado o proceso.	Actualiza los resultados combinando múltiples criterios de selección.	Combinación sin registros asociados.
4	Seleccionar un postulante específico de la tabla.	Navega hacia la ficha integral del candidato.	Postulante no disponible o eliminado.
5	Revisar la ficha completa y su trazabilidad.	Muestra datos de contacto, documentos asociados, historial y estados.	Sección con error de carga parcial.

Principios de UX y accesibilidad aplicados

Control de Accesibilidad	Criterio práctico aplicado	Componentes de UI involucrados
Contraste de texto	Relación mínima de 4.5:1 para texto normal y 3:1 para texto grande.	Tokens de color compartidos en Tailwind CSS.
Tamaño de objetivo táctil	Mínimo de 24x24 px; preferible 44x44 px en elementos interactivos críticos.	Botones de acción y controles en PostulantesTable.
Foco visible	Todo control interactivo muestra un indicador claro con :focus-visible.	Inputs, botones y filas navegables por teclado.
Uso de color no exclusivo	Los estados de error, éxito o selección incorporan texto descriptivo o iconos.	EstadoBadge y mensajes de validación en formularios.
Ayuda y gestión de errores	Mensajes precisos que identifican el campo afectado y orientan en la corrección.	Formularios de registro (Postulantes, Empresas, Anuncios).